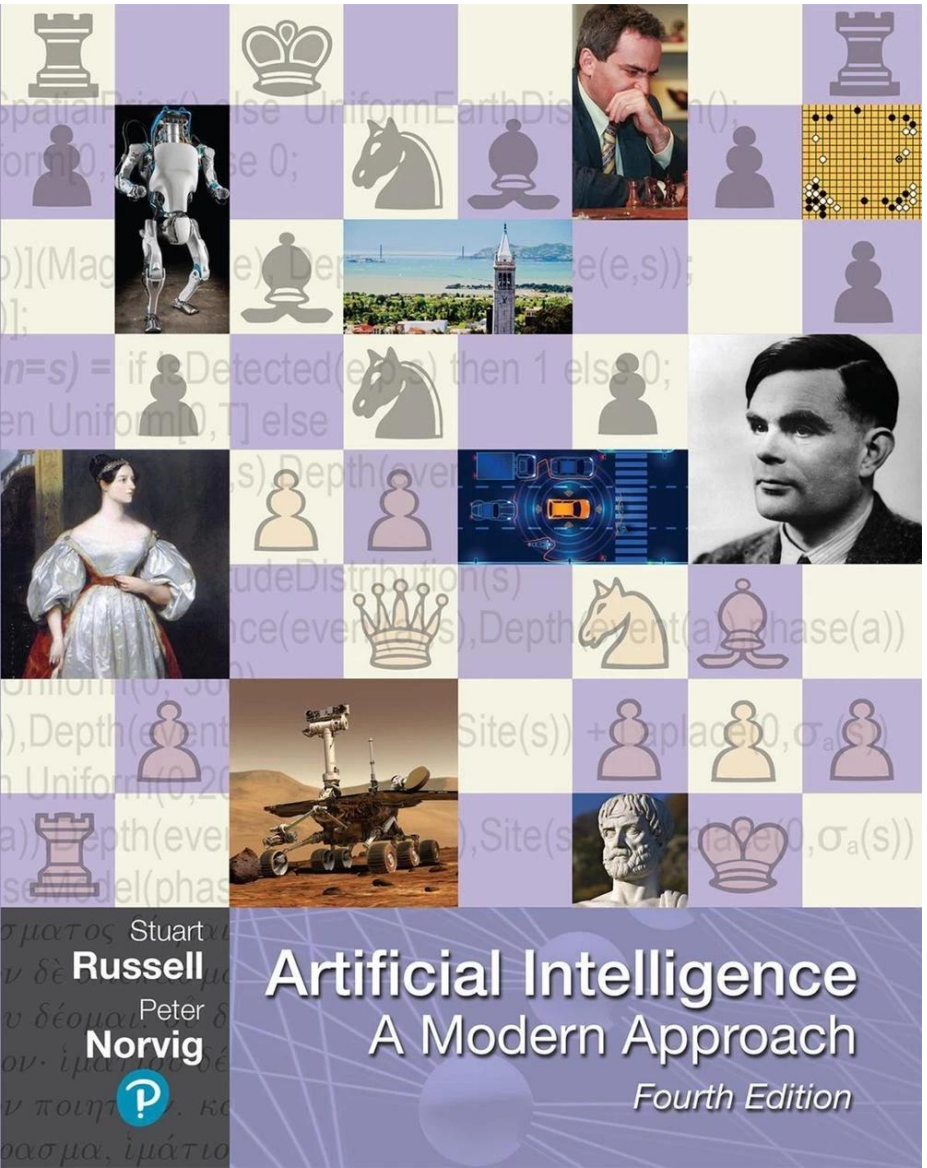
HOA SEN UNIVERSITY

www.hoasen.edu.vn

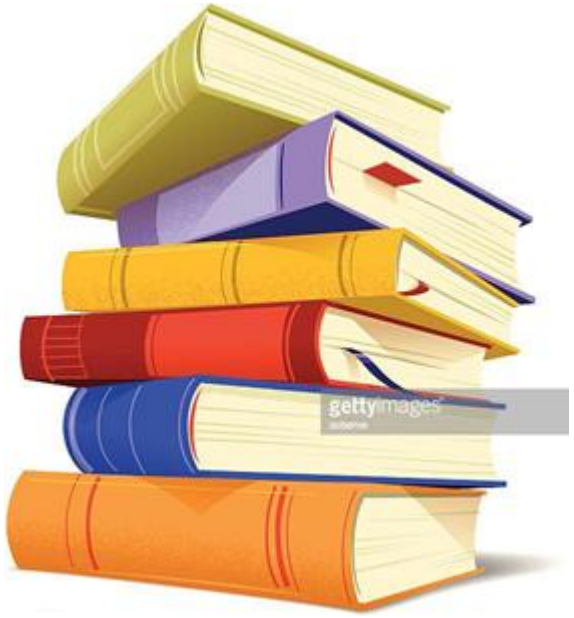


Lecture 1

Solving Problems by Searching (Part I)

HOA SEN UNIVERSITY

Solving Problems by Searching (Part I)



- Problem-Solving Agents
- Search Algorithms
- Uninformed (Blind) Search Strategies

HOA SEN UNIVERSITY

www.hoasen.edu.vn

Problem-Solving Agents

HOA SEN UNIVERSITY

Problem-Solving Agents

- Agents can follow this four-phase problem-solving process:
  - **Goal Formulation:** Based on the current situation.
  - **Problem Formulation:** Deciding what states and actions to consider.
  - **Search:** Simulating sequences of actions in the model.
  - **Execution:** Carrying out the actions.

HOA SEN UNIVERSITY

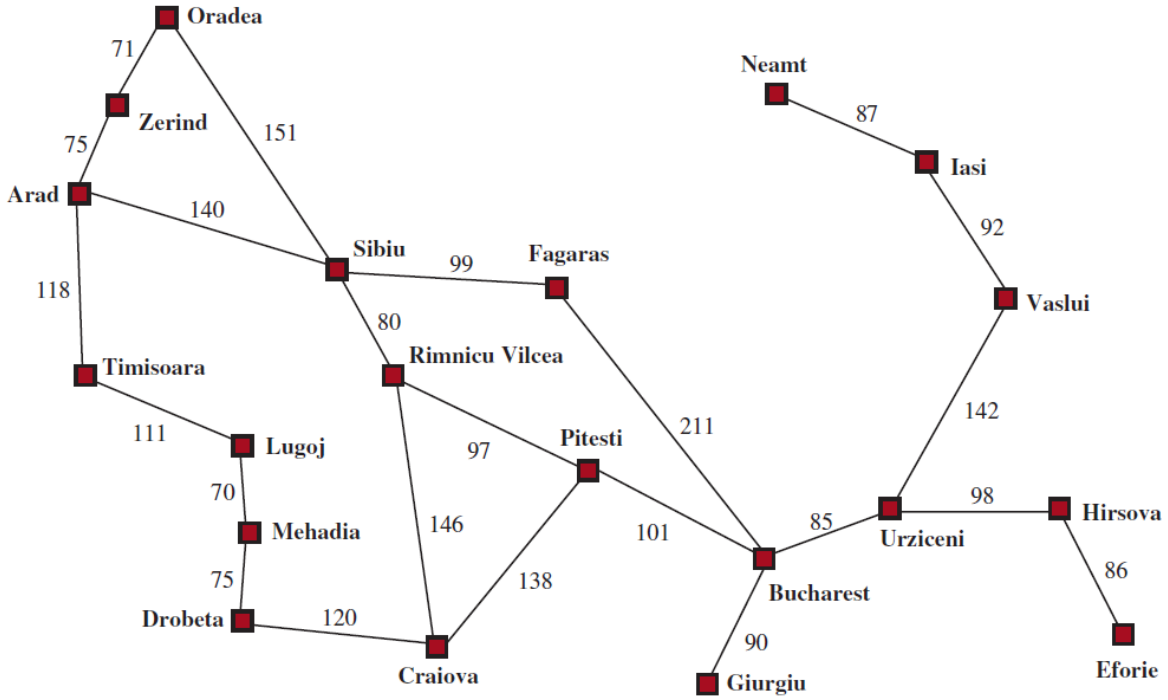
Defining a Search Problem

- **A search problem consists of five components:**
  - **Initial State:** The starting point.
  - **Actions:** Set of possible actions at each state.
  - **Transition Model:** RESULT(s, a) returns the state resulting from action **a** in state **s**.
  - **Goal States:** A set of target states.
  - **Path cost:** The sum of step costs (e.g., distance).

HOA SEN UNIVERSITY

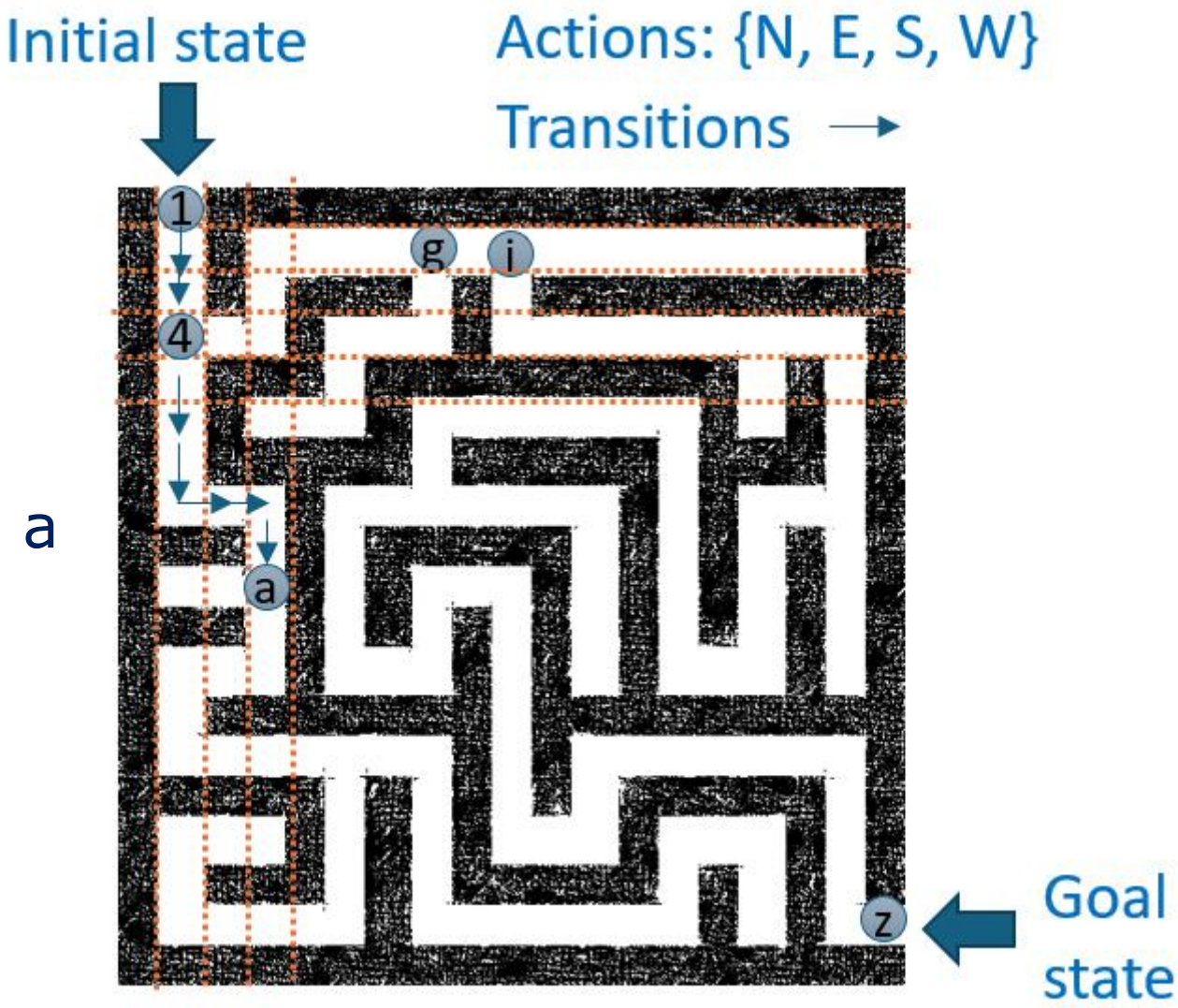
Example: Romania Vacation

- On vacation in Romania, currently in Arad. Flight leaves tomorrow from Bucharest.
- **Initial State:** Arad
- **Actions:** Drive from one city to another
- **Transition Model:** If you go from city A to city B, you end up in city B
- **Goal States:** Bucharest
- **Path cost:** Sum of edge costs



Example: Maze

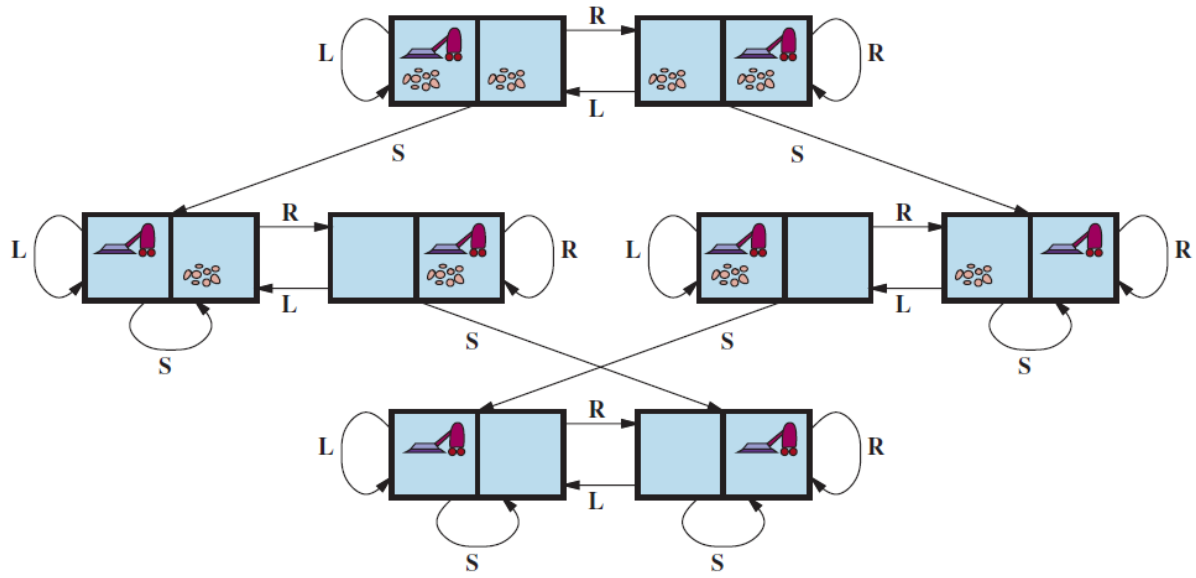
- **Initial State:** Position **1**
- **Actions:** N, E, S, W
- **Transition Model:** Move from a location to another
- **Goal States:** Position **z**
- **Path cost:** Number if actions



HOA SEN UNIVERSITY

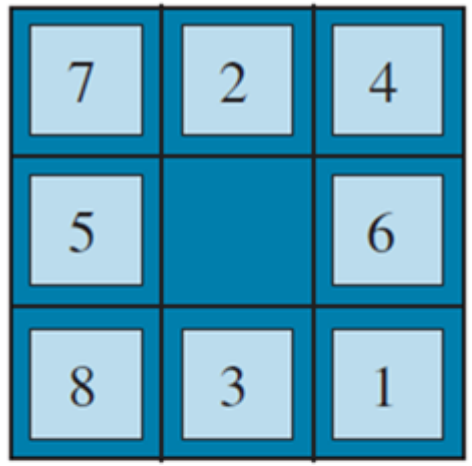
Example: The two-cell vacuum world

- **Initial State:** Defined by agent location and dirt location
- **Actions:** Left, right, suck
- **Transition Model:** Clean a location or move
- **Goal States:** All locations are clean
- **Path cost:** Number if actions

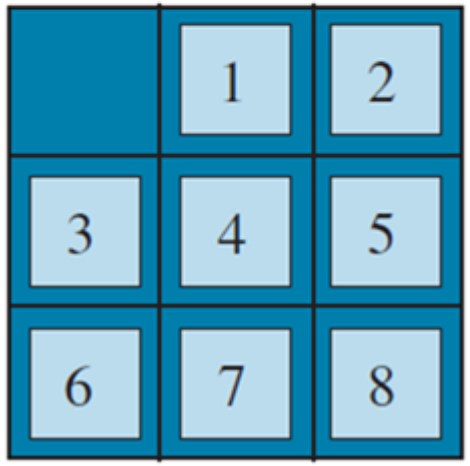


Example: The 8-puzzle

- **Initial State:** A given configuration
- **Actions:** Move blank left, right, up, down
- **Transition Model:** Move a tile
- **Goal States:** Tiles are arranged empty and 1-8 in order
- **Path cost:** 1 per tile move



Start State



Goal State

HOA SEN UNIVERSITY

Example Problems: Toy vs. Real-World

- **Standardized (Toy) Problems:** Used as benchmarks for researchers (Concise and exact).
  - *Vacuum World, 8-puzzle, Sokoban, Knuth's 4-problem.*
- **Real-World Problems:** Solutions that people actually use (Idiosyncratic).
  - *Route finding, Airline travel, VLSI layout, Robot navigation.*

HOA SEN UNIVERSITY

www.hoasen.edu.vn

Search Algorithms

HOA SEN UNIVERSITY

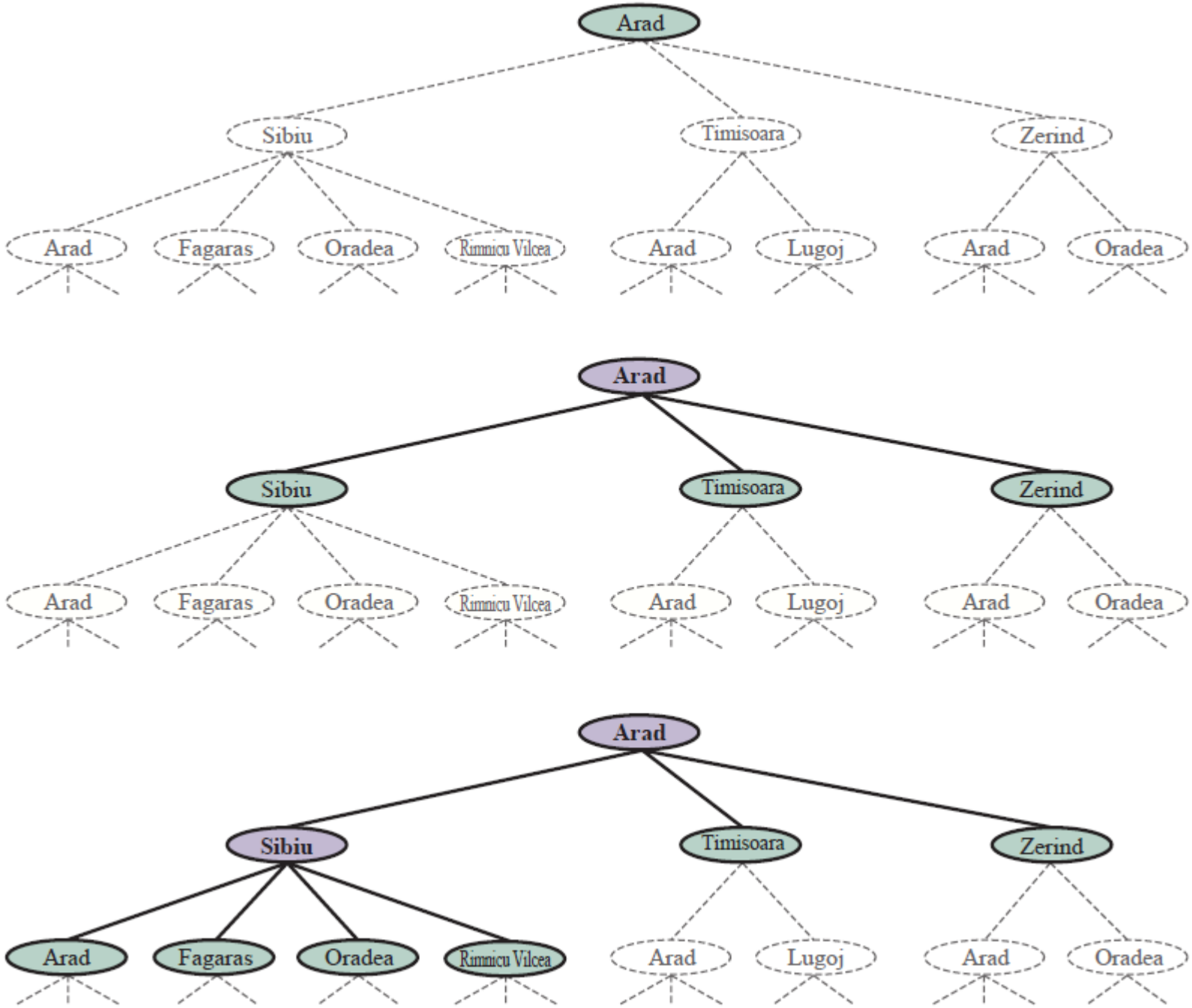
Search Trees

- Algorithms superimpose a search tree over the state-space graph.
- **Node:** A data structure in the tree (State, Parent, Action, Path-cost).
- **Frontier:** The set of all nodes generated but not yet expanded.
- **Expansion:** Applying all applicable actions to a node to generate children.

HOA SEN UNIVERSITY

Search Trees

- Three partial search trees for finding a route from Arad to Bucharest



The Best-First Search Algorithm

- **A general framework for search where the node with the minimum $f(n)$ is expanded.**
- **Mechanism:** Repeatedly choose a node from the frontier with minimum evaluation function $f(n)$.
- **Redundant Paths:** Handled by the **reached** lookup table to avoid repeating the past.
- **Queue Types:** Priority queue (for best-first), FIFO queue, or LIFO queue.

HOA SEN UNIVERSITY

Measuring Performance

- **Four criteria to evaluate search algorithms:**
  - **Completeness:** Is it guaranteed to find a solution if one exists?
  - **Cost Optimality:** Does it find the lowest-path cost among all solutions?
  - **Time Complexity:** How long does it take to find a solution?
  - **Space Complexity:** How much memory is needed for the search?

HOA SEN UNIVERSITY

www.hoasen.edu.vn

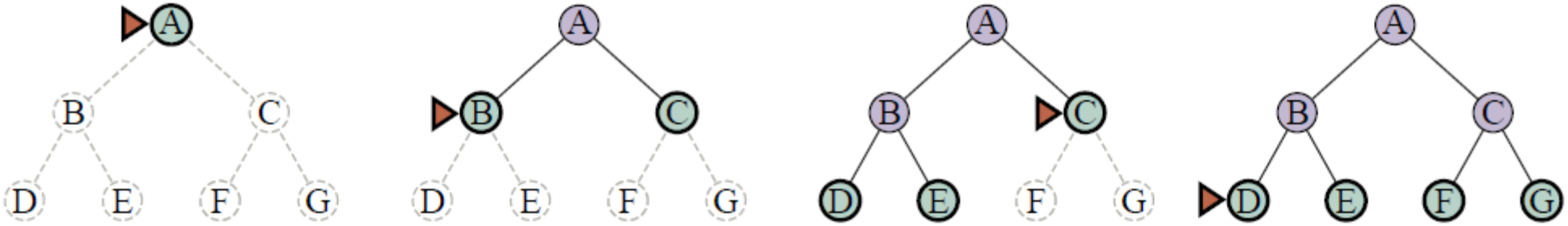
Uninformed (Blind) Search Strategies

HOA SEN UNIVERSITY

b: maximum branching factor
m: max depth of tree
d: depth of the optimal solution

Breadth-First Search (BFS)

- **Expands the root node first, then all successors at each level.**
  - **Frontier:** FIFO (First-In, First-Out) queue.
  - **Completeness:** Complete (if branching factor b is finite).
  - **Optimality:** Optimal if all action costs are identical.
  - **Complexity:** Time and Space are both $O(b^d)$.



HOA SEN UNIVERSITY

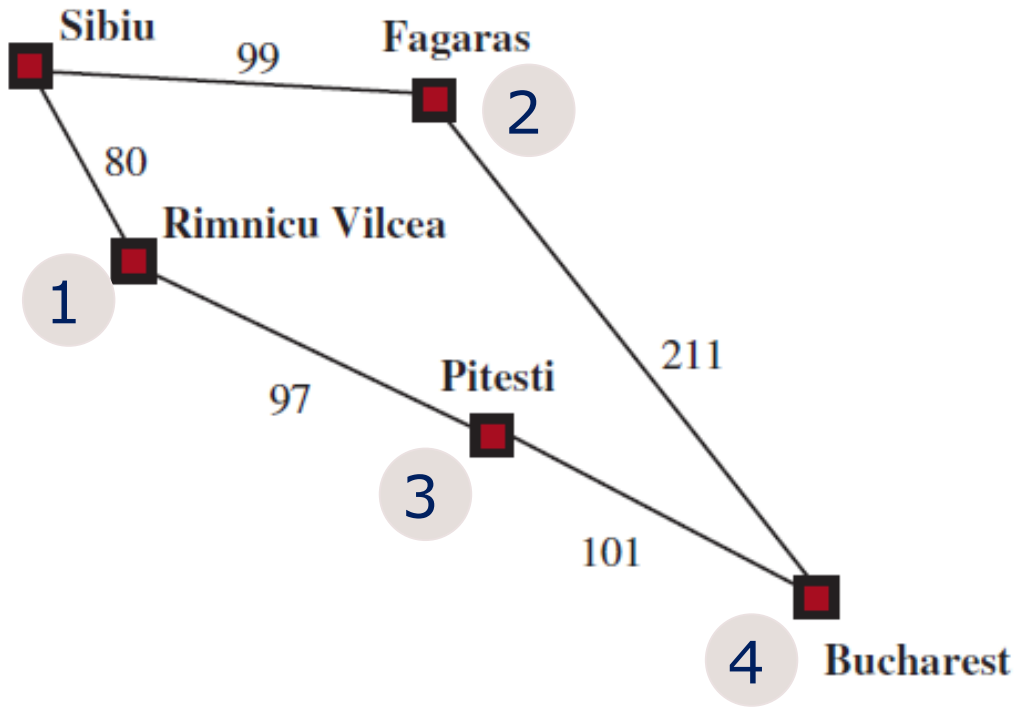
Uniform-Cost Search (UCS)

- **Expands the node n with the lowest path cost $g(n)$.**
  - **Algorithm:** Also known as Dijkstra's algorithm.
  - **Frontier:** Priority queue ordered by g.
  - **Goal Test:** Applied when a node is expanded.
  - **Optimality:** Optimal for any action costs > 0.

HOA SEN UNIVERSITY

Uniform-Cost Search (UCS)

- The problem is to get from Sibiu to Bucharest
- Sibiu → Rimnicu Vilcea (cost = 80)
- Sibiu → Fagaras (cost = 90)
- Rimnicu Vilcea → Pitesti (cost = 80+97=177)
- Pitesti → Bucharest (cost 80+97+101=278)



HOA SEN UNIVERSITY

b: maximum branching factor
m: max depth of tree
d: depth of the optimal solution

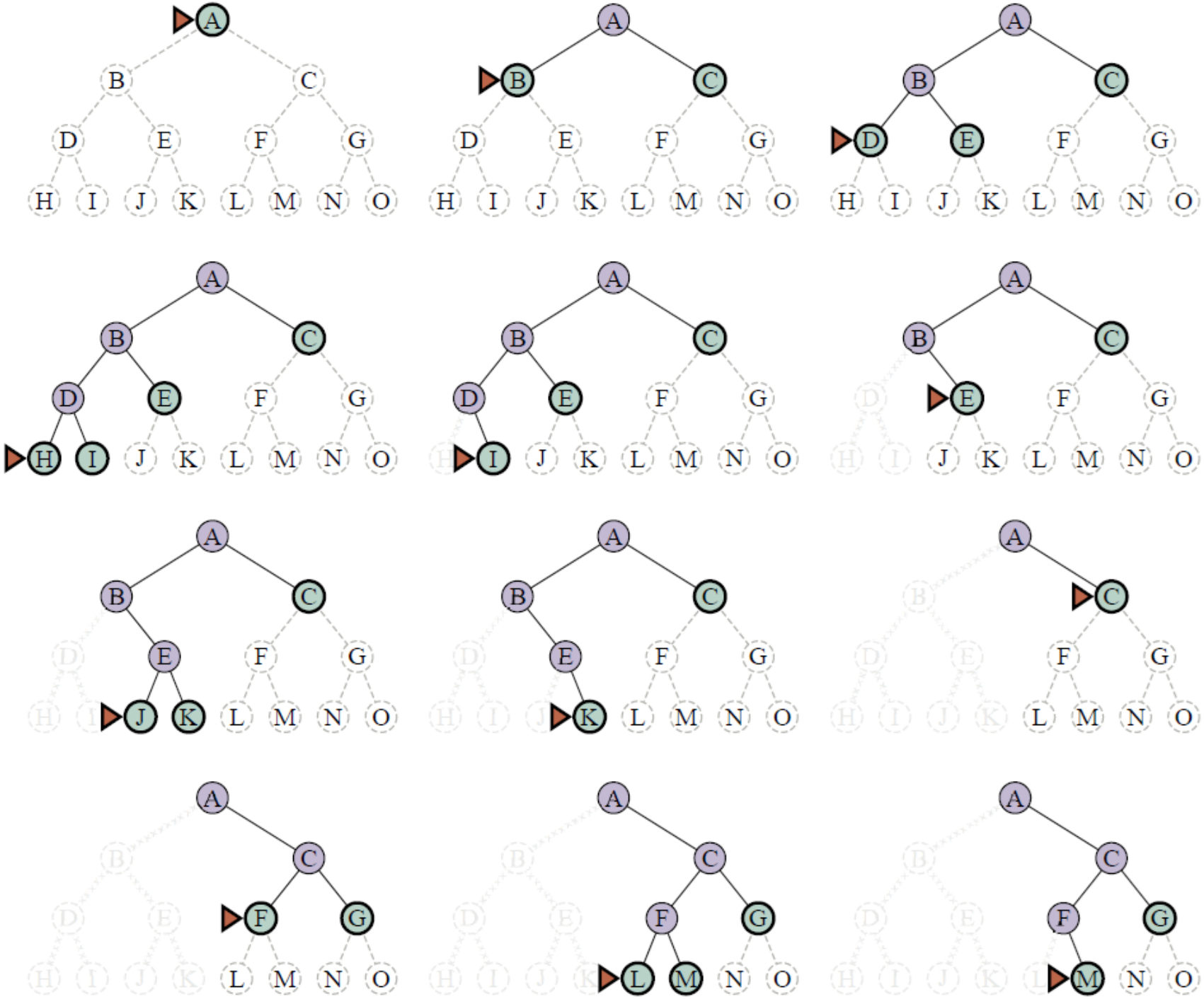
Depth-First Search (DFS)

- **Always expands the deepest node in the current frontier.**
  - **Frontier:** LIFO (Last-In, First-Out) queue or Stack.
  - **Space Complexity:** $O(bm)$ - much more efficient than BFS.
  - **Completeness:** Not complete in infinite spaces or with cycles.
  - **Optimality:** Not optimal.

HOA SEN UNIVERSITY

DFS

- The progress of a DFS from start state A to goal M.
- The frontier is in green, with a triangle marking the node to be expanded next.
- Expanded nodes with no descendants in the frontier can be discarded.



HOA SEN UNIVERSITY

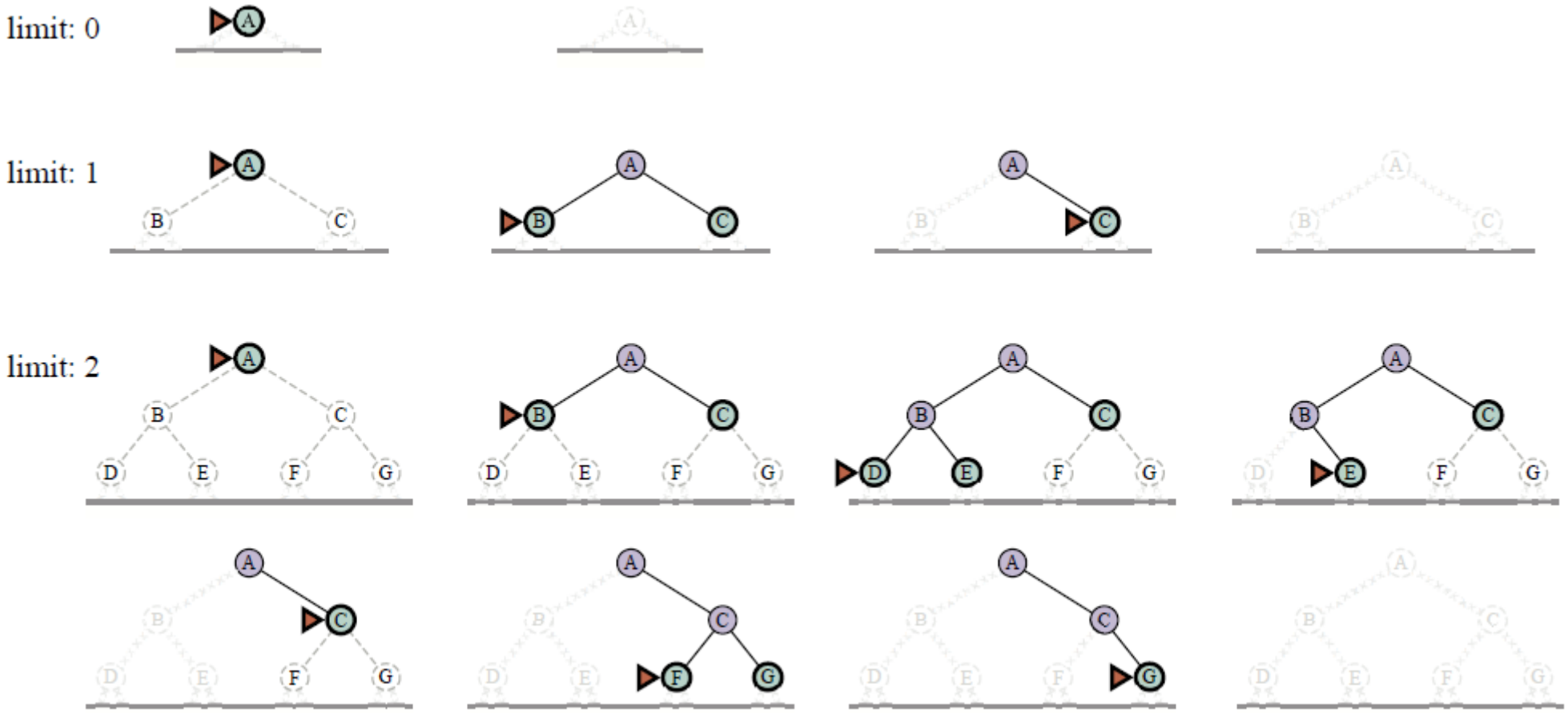
b: maximum branching factor
m: max depth of tree
d: depth of the optimal solution

Iterative Deepening Search (IDS)

- **Combines BFS's optimality and completeness with DFS's space efficiency.**
  - **Mechanism:** Gradually increases the depth limit (0, 1, 2, ...) until a goal is found.
  - **Preferred Method:** The preferred uninformed search for large state spaces.
  - **Time Complexity:** $O(b^d)$ - similar to BFS.

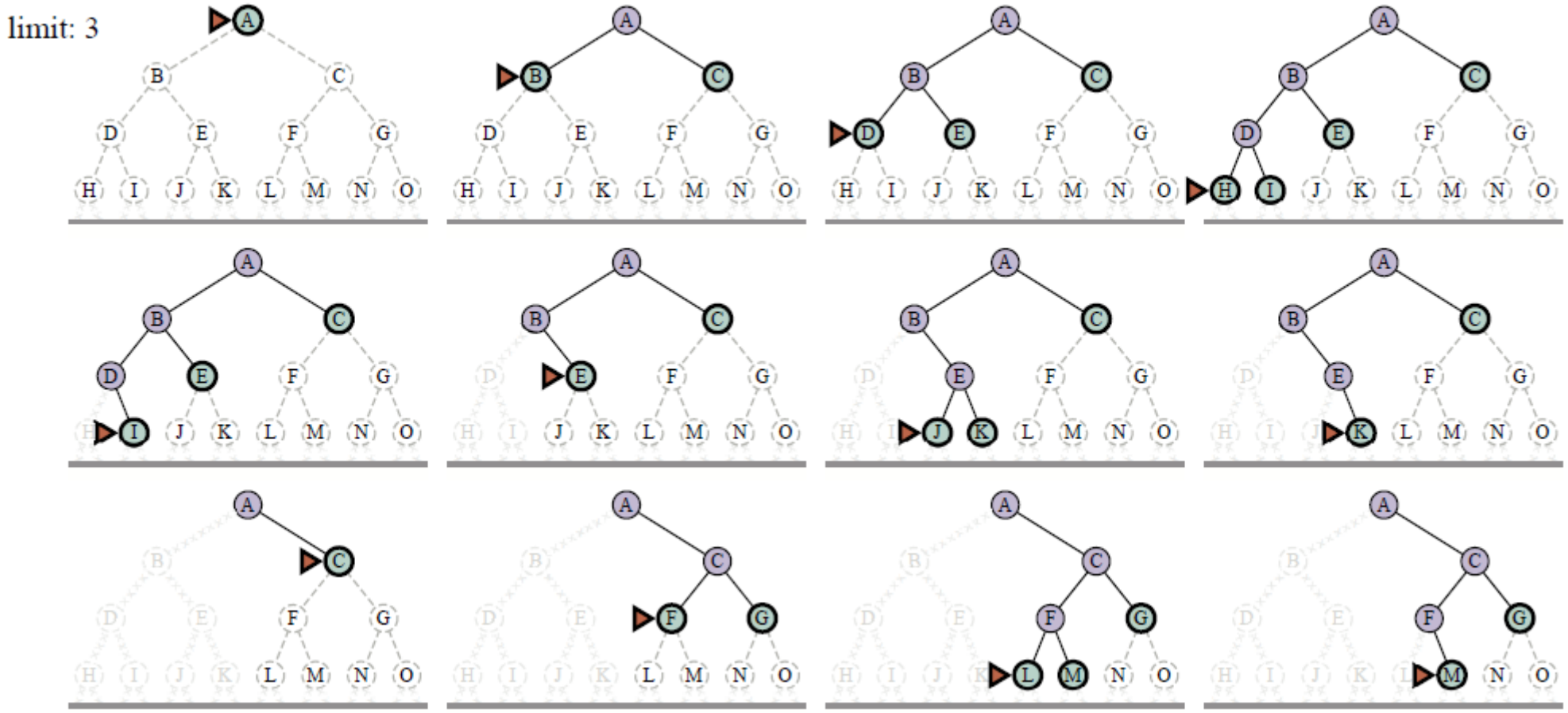
HOA SEN UNIVERSITY

Iterative Deepening Search (IDS)



HOA SEN UNIVERSITY

Iterative Deepening Search (IDS)

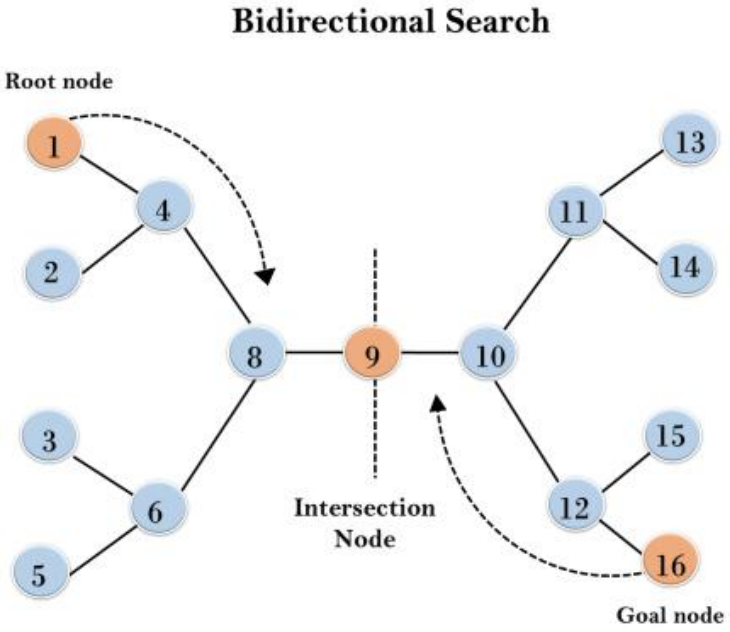


HOA SEN UNIVERSITY

b: maximum branching factor
m: max depth of tree
d: depth of the optimal solution

Bidirectional Search

- **Searches forward from the start and backward from the goal simultaneously.**
  - **Goal:** The two searches meet in the middle.
  - **Advantage:** $b^{d/2} + b^{d/2}$ is much smaller than b^d.
  - **Challenges:** How to search backward? What if there are multiple goal states?



HOA SEN UNIVERSITY

Comparison of Uninformed Search Strategies

Strategy	Complete	Optimal	Time	Space
BFS	Yes	Yes*	$O(b^d)$	$O(b^d)$
UCS	Yes	Yes	$O(b^{1+\lfloor C^*/\epsilon \rfloor})$	$O(b^{1+\lfloor C^*/\epsilon \rfloor})$
DFS	No	No	$O(b^m)$	$O(bm)$
IDS	Yes	Yes*	$O(b^d)$	$O(bd)$
Bidirectional	Yes	Yes*	$O(b^{d/2})$	$O(b^{d/2})$

**Optimal if all step costs are equal.*

HOA SEN UNIVERSITY

Summary

- **Four-phase Process:** Goal formulation, Problem formulation, Search, and Execution.
- **A well-defined problem has five components:**
  - **Initial State:** Starting point.
  - **Actions:** Available choices.
  - **Transition Model:** Results of actions.
  - **Goal States:** Target conditions.
  - **Action Cost Function:** Numeric path cost.
- **Problem Types:** Standardized (toy) problems for benchmarking vs. complex real-world applications.

HOA SEN UNIVERSITY

Summary

- **Search Tree vs. State Space:** Algorithms expand nodes to form a tree; the frontier separates explored from unexplored regions.
- **Performance Metrics:** Completeness, Optimality, Time complexity $O(b^d)$, and Space complexity.
- **Uninformed (Blind) Search Strategies:**
  - **BFS (Breadth-First):** FIFO, shallowest first. Optimal for uniform cost.
  - **UCS (Uniform-Cost):** Priority queue by $g(n)$. Optimal for any cost.
  - **DFS (Depth-First):** LIFO (stack), deepest first. Memory efficient $O(bm)$.
  - **IDS (Iterative Deepening):** Preferred for large spaces; combines BFS benefits with DFS memory.
  - **Bidirectional:** Searches from both ends; reduces time to $O(b^{d/2})$.

HOA SEN UNIVERSITY